

Principles of Distributed Systems

第7周：一致性与复制
Week 7: Consistency & Replication

By: Fahad Sabah

计算机学院,
北京工业大学, 中国
College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



一致性与复制

Consistency & Replication

- ❑ Introduction (what's it all about)
- ❑ Data-centric consistency
- ❑ Client-centric consistency
- ❑ Replica management
- ❑ Consistency protocols
- ❑ 介绍（这到底是怎么回事）
- ❑ 数据中心一致性
- ❑ 以客户端为中心的一致性
- ❑ 副本管理
- ❑ 一致性协议

介绍（这到底是怎么回事）

Introduction (what's it all about)

A very easy analogy: multiple copies of the same notebook

Imagine a class has 3 shared notebooks in different locations:

Notebook A

Notebook B

Notebook C

All three are supposed to contain the same information.

If one student writes in Notebook A, the other notebooks should eventually show the same update too. That's replication.

一个非常简单的比喻：同一本笔记本的多份副本

想象一个班级有3本共用笔记本，分布在不同位置：

笔记本A

笔记本B

笔记本C

这三者应该包含相同的信息。

如果一个学生在笔记本A里写，其他笔记本最终也会显示同样的更新。这就是复制。

介绍（这到底是怎么回事）

Introduction (what's it all about)

Why consistency is hard

The problem appears when two people try to change the same thing at the same time.

Example 1: write–write conflict

Two students update the same class roster:

- ❑ Student 1 writes: “Alice = present”
- ❑ Student 2 writes: “Alice = absent”

If Notebook A sees Student 1 first and Notebook B sees Student 2 first, the notebooks may disagree unless we force the same order everywhere.

Example 2: read–write conflict

One student is reading the roster while another is updating it:

- ❑ Reader checks: “Is Alice present?”
- ❑ Writer changes Alice’s status from “absent” to “present”

If the read happens before the write in one place, but after the write in another place, different replicas may return different answers.

介绍（这到底是怎么回事）

Introduction (what's it all about)

Key idea: conflicting operations must be ordered the same way everywhere

关键思想：冲突操作必须在各地以相同方式排序

“If two operations can affect each other, all replicas must agree on which one happened first.”

“如果两个操作能互相影响，所有复制体必须同意哪个先发生。”

介绍（这到底是怎么回事）

Introduction (what's it all about)

Why strong global ordering is expensive

To guarantee the same order everywhere, replicas may need:

- ☐ coordination
- ☐ messaging
- ☐ waiting for acknowledgments
- ☐ consensus protocols

That adds delay and reduces scalability.

“The more we force everyone to agree immediately, the more communication and waiting we introduce.”

为什么强有力的全球秩序代价高昂

为了保证所有地方的顺序一致，副本可能需要：

- ☐ 协调
- ☐ 信息传递
- ☐ 等待确认
- ☐ 共识协议

这会增加延迟并降低可扩展性。

“我们越是强迫大家立刻同意，沟通和等待就越多。”

介绍（这到底是怎么回事）

Introduction (what's it all about)

Why systems weaken consistency

So many distributed systems choose a tradeoff:

Strong consistency: easier to reason about, but slower and less scalable

Weaker consistency: faster and more scalable, but replicas may temporarily differ

This is the main design choice:

“Do we want all copies to look identical right now, or is it okay if they converge later?”

为什么系统会削弱一致性

许多分布式系统选择权衡：

强一致性：更容易推理，但速度较慢且扩展性较低

较弱的一致性：速度更快且更具可扩展性，但复制品可能暂时存在差异

这是主要的设计选择：

“我们现在是想让所有复制品看起来都一模一样吗，还是说它们以后会合并也没关系？”

介绍（这到底是怎么回事）

Introduction (what's it all about)

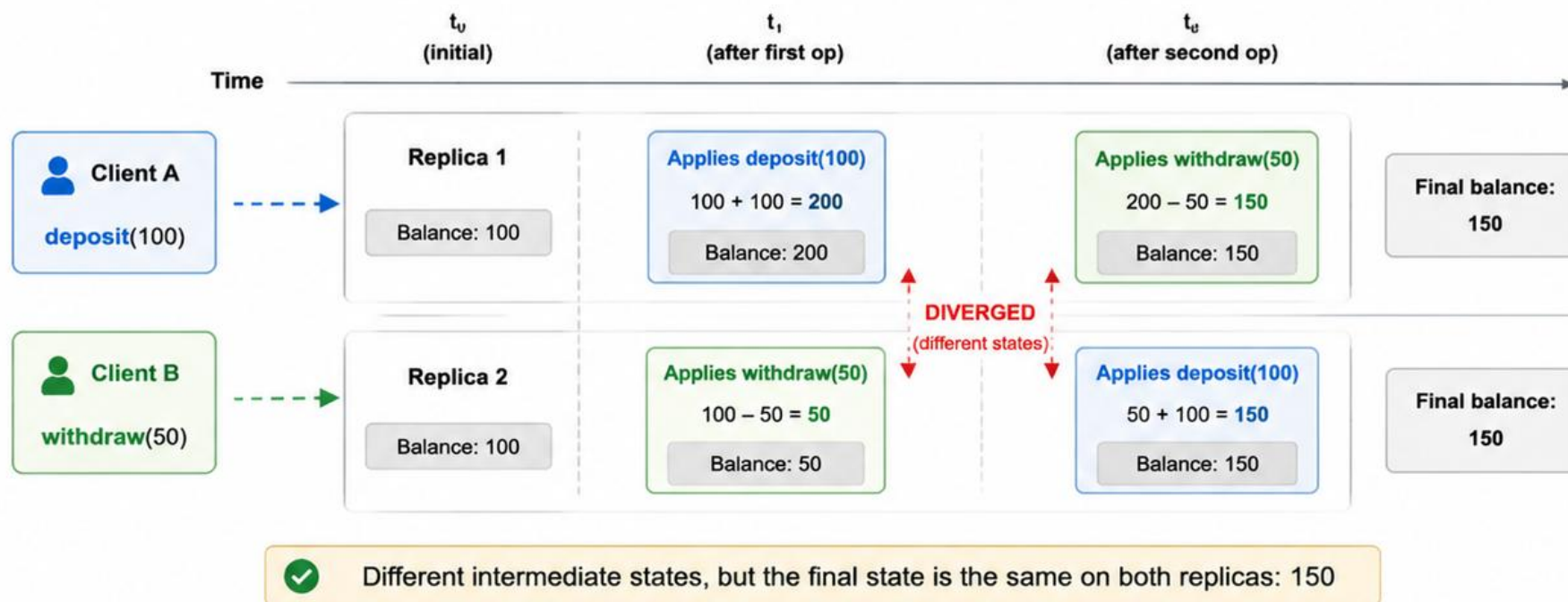
Replication and Consistency

- ❑ Replication = keeping multiple copies of data
- ❑ Consistency = making sure copies don't contradict each other
- ❑ Problem: concurrent conflicting operations
 - ❑ read-write conflict
 - ❑ write-write conflict
- ❑ Requirement: all conflicting operations must be ordered the same way everywhere
- ❑ Cost: global ordering needs coordination, which hurts scalability
- ❑ Solution: weaken consistency to reduce synchronization

介绍 (这到底是怎么回事)

Introduction (what's it all about)

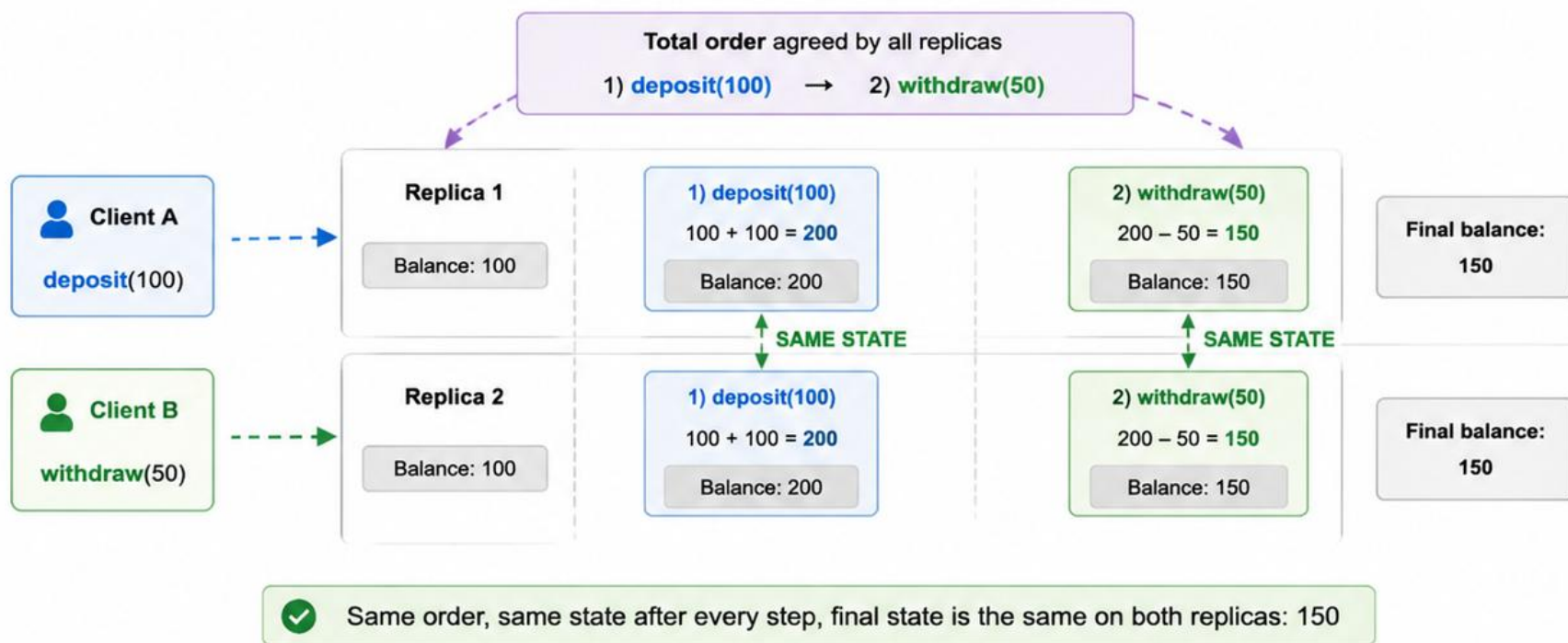
1) Without total ordering: replicas diverge temporarily but end up the same



介绍 (这到底是怎么回事)

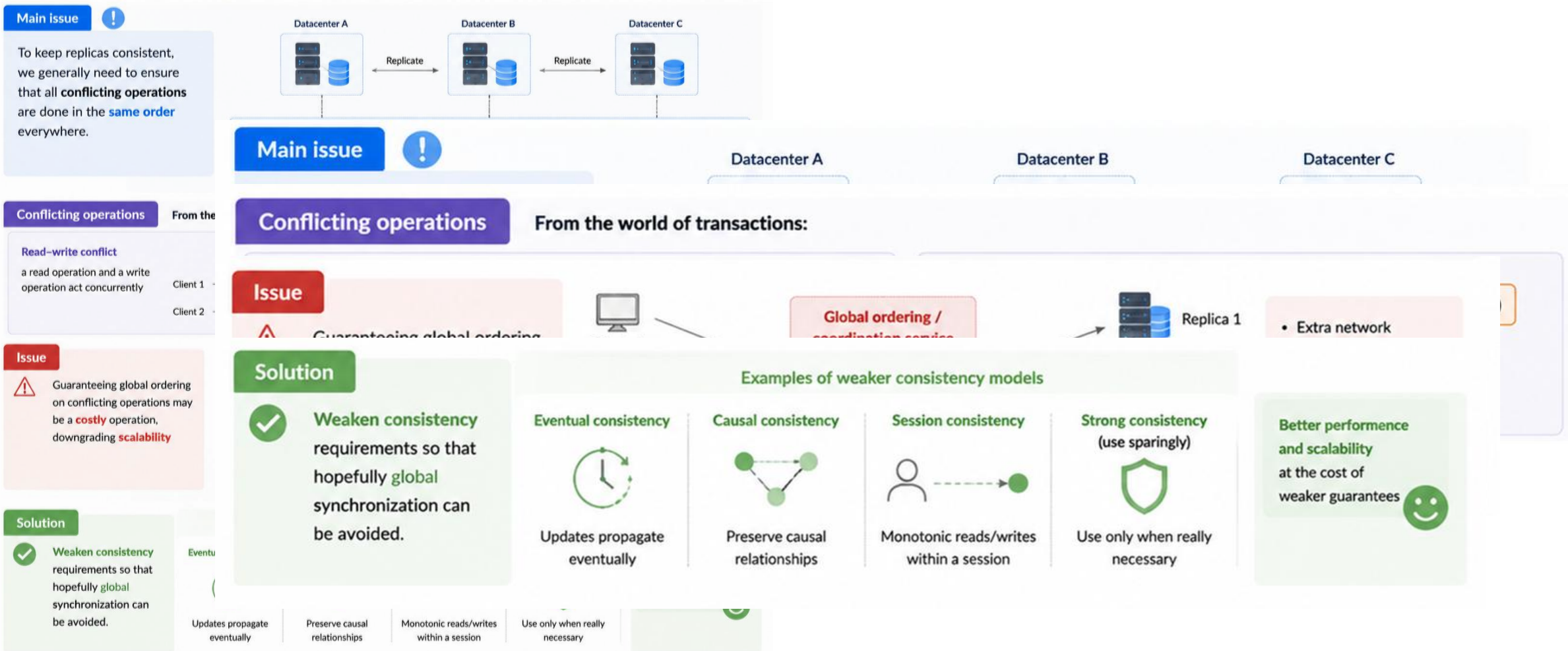
Introduction (what's it all about)

2) With total ordering: replicas execute the same order and stay identical at every step



介绍 (这到底是怎么回事)

Introduction (what's it all about)



以数据为中心的一致性模型

Data-centric consistency models

Consistency model

A contract between a (distributed) data store and processes, in which the data store specifies precisely what the results of read and write operations are in the presence of concurrency.

Essential

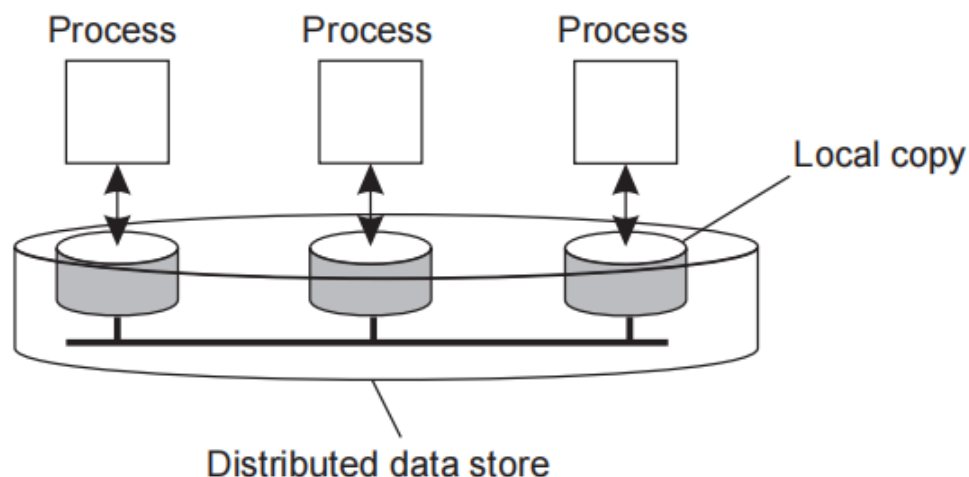
A data store is a distributed collection of storages

一致性模型

一个（分布式）数据存储与进程之间的契约，数据存储并发情况下精确指定读写操作的结果。

必备

数据存储是分布式存储集合



以数据为中心的一致性模型

Data-centric consistency models

Continuous Consistency

Observation

We can actually talk about a degree of consistency:

- ❑ replicas may differ in their numerical value
- ❑ replicas may differ in their relative staleness
- ❑ there may be differences with respect to (number and order) of performed update operations

Conit

Consistency unit $\Rightarrow$ specifies the data unit over which consistency is to be measured.

连续一致性

观察

我们实际上可以谈谈某种程度的一致性：

- ❑ 复制品的数值可能不同
- ❑ 复制品可能在相对陈旧程度上有所不同
- ❑ 在执行的更新操作数量和顺序上可能存在差异

Conit

一致性单位 $\Rightarrow$ 指定要测量一致性的数据单位。

以数据为中心的一致性模型

Data-centric consistency models

Example: Conit

Think of it as:

Two replicas (copies of data) exist:

Replica A

Replica B

Both store the same variables: x and y

Updates happen independently and may not arrive immediately at the other replica. So temporarily, the replicas can become different.

1. What is happening here?

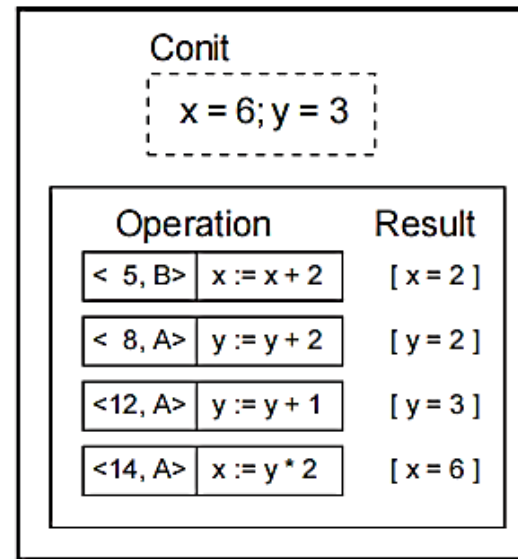
Initially both replicas probably started with:

$x = 0, y = 0$

Then updates happen separately.

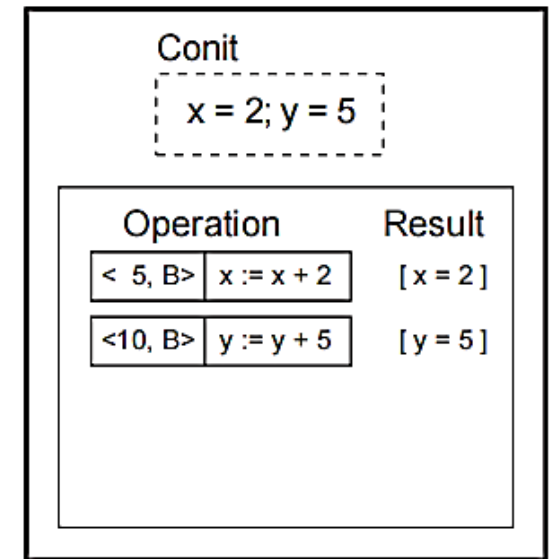
This figure is showing eventual consistency with bounded divergence called a conit (consistency unit).

Replica A



Vector clock A = (15, 5)
Order deviation = 3
Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)
Order deviation = 2
Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

Example: Conit

2. Replica A operations

Replica A has received these updates:

Time	Operation	New Value
<5,B>	$x := x + 2$	$x = 2$
<8,A>	$y := y + 2$	$y = 2$
<12,A>	$y := y + 1$	$y = 3$
<14,A>	$x := y * 2$	$x = 6$

Final state at Replica A: $x = 6$ $y = 3$

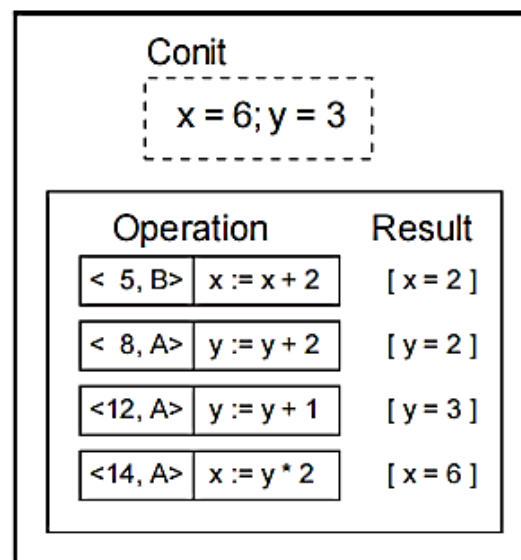
3. Replica B operations

Replica B has only received:

Time	Operation	New Value
<5,B>	$x := x + 2$	$x = 2$
<10,B>	$y := y + 5$	$y = 5$

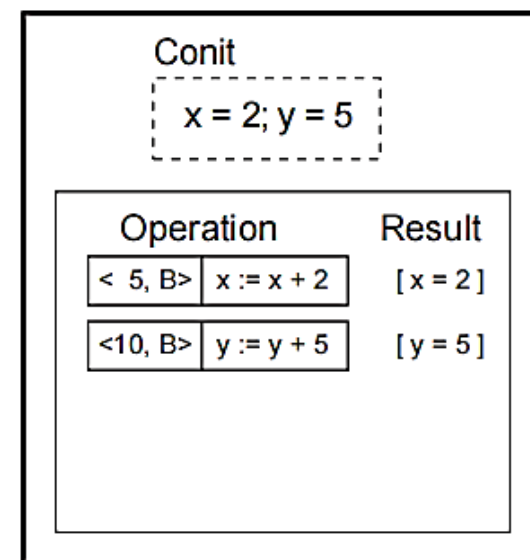
Final state at Replica B: $x = 2$ $y = 5$

Replica A



Vector clock A = (15, 5)
Order deviation = 3
Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)
Order deviation = 2
Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

Example: Conit

4. Why are replicas different?

Because not all updates have been exchanged yet.

Replica A knows about:

$y := y + 2$

$y := y + 1$

$x := y * 2$

But Replica B has not received them yet.

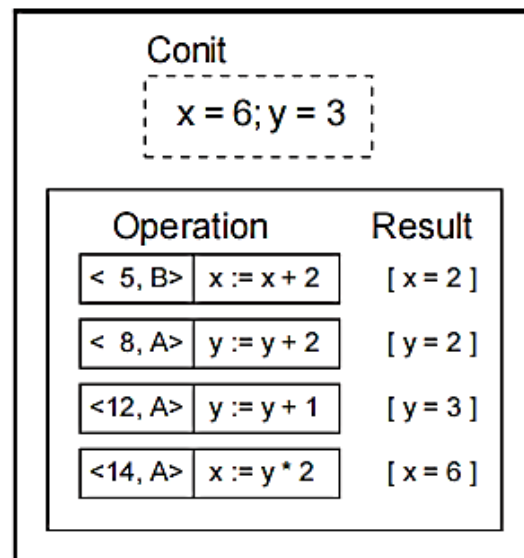
Replica B knows about:

$y := y + 5$

But Replica A may not have fully incorporated that operation.

This is normal in weak/eventual consistency systems.

Replica A

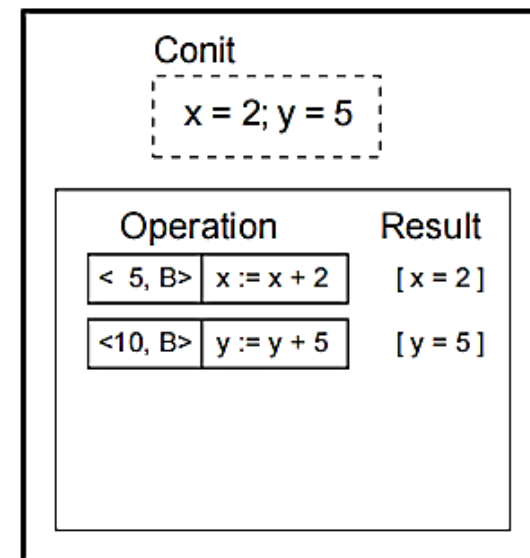


Vector clock A = (15, 5)

Order deviation = 3

Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)

Order deviation = 2

Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

5. Easy understanding of vector clocks

Vector clocks track:

“How many updates from each replica have I seen?”

Format:

(A_updates, B_updates)

Replica A vector clock (15,5)

Meaning:

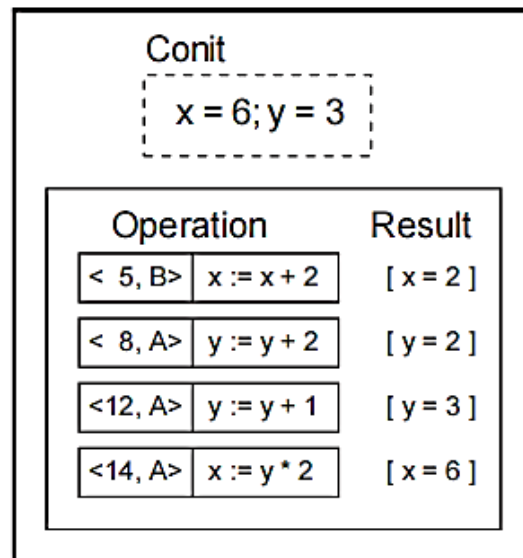
- ❑ Replica A has seen A's updates up to logical time 15
 - ❑ Replica A has seen B's updates up to logical time 5
- So A is missing newer updates from B.

Replica B vector clock (0,11)

Meaning:

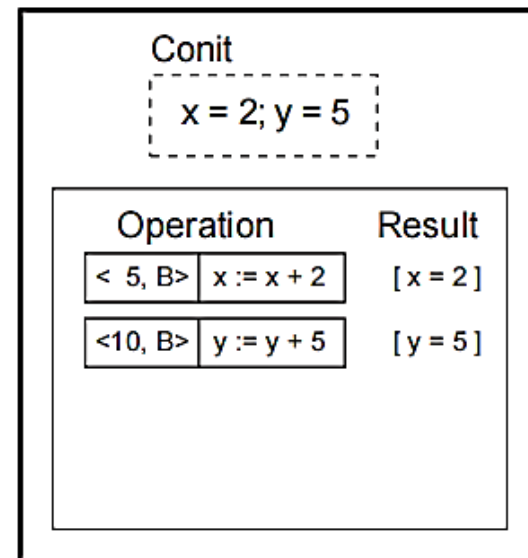
- ❑ Replica B has not yet seen A's newer updates
 - ❑ Replica B has seen B's updates up to time 11
- So B is missing A's updates.

Replica A



Vector clock A = (15, 5)
Order deviation = 3
Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)
Order deviation = 2
Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

6. What is Order Deviation?

Order deviation tells:

“How many updates are still missing?”

Replica A

Order deviation = 3

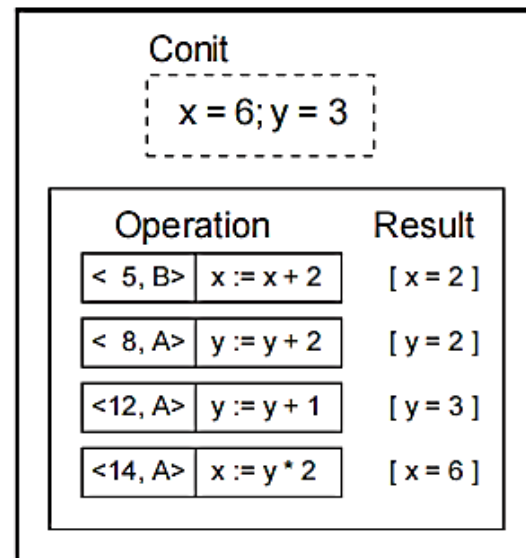
Replica A is missing 3 operations from Replica B.

Replica B

Order deviation = 2

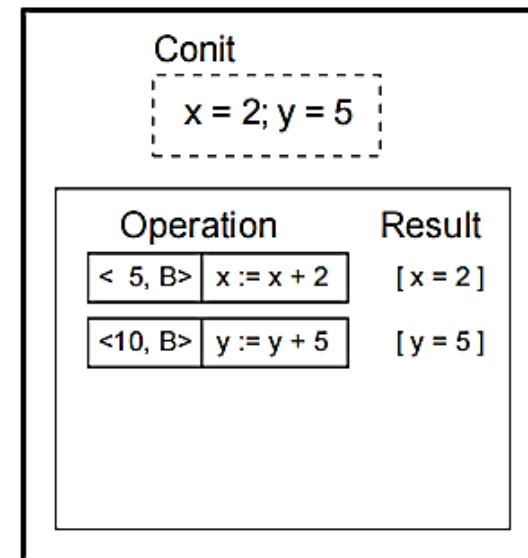
Replica B is missing 2 operations from Replica A.

Replica A



Vector clock A = (15, 5)
Order deviation = 3
Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)
Order deviation = 2
Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

Example: Conit

7. What is Numerical Deviation?

This measures:

“How different are the actual data values?”

Replica A

Numerical deviation = (1,5)

Meaning:

x differs by 1 from the fully synchronized value

y differs by 5

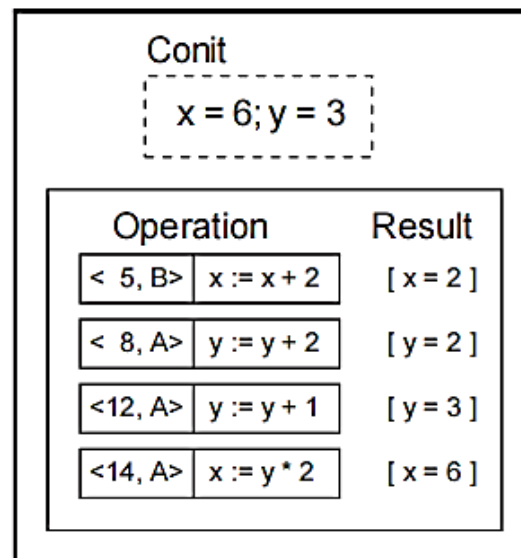
Replica B

Numerical deviation = (3,6)

Meaning

Replica B is even further from the final correct values.

Replica A

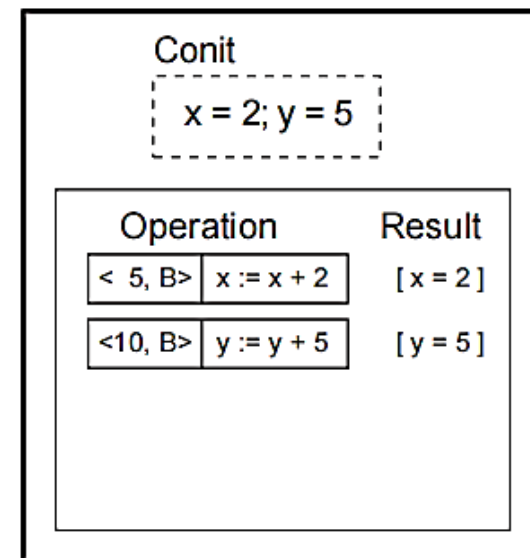


Vector clock A = (15, 5)

Order deviation = 3

Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)

Order deviation = 2

Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

Example: Conit

8. The main idea of the figure

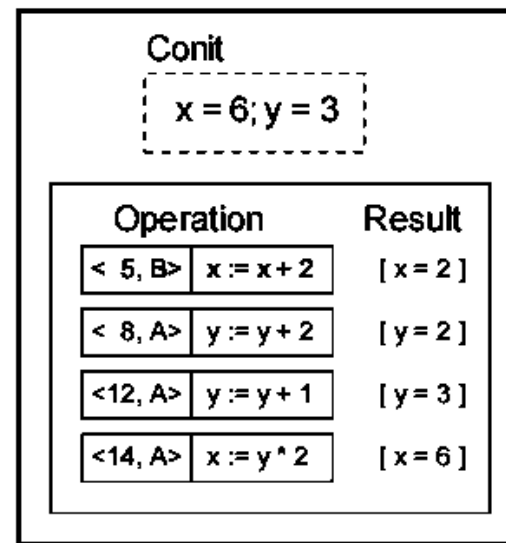
- ☐ Replicas can temporarily diverge
- ☐ Updates may arrive at different times
- ☐ Systems may tolerate some inconsistency
- ☐ Eventually replicas exchange updates and converge again

This improves:

- ☐ Scalability
- ☐ Availability
- ☐ Performance

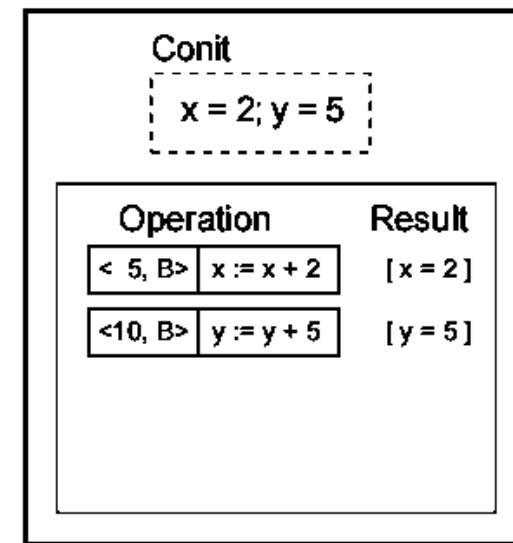
because replicas do not need global synchronization after every operation.

Replica A



Vector clock A = (15, 5)
Order deviation = 3
Numerical deviation = (1, 5)

Replica B



Vector clock B = (0, 11)
Order deviation = 2
Numerical deviation = (3, 6)

以数据为中心的一致性模型

Data-centric consistency models

First, the notation:

- ❑ $W(x)a \rightarrow$ write value a into variable x
- ❑ $W(x)b \rightarrow$ write value b into variable x
- ❑ $R(x)a \rightarrow$ read x and get value a
- ❑ $R(x)b \rightarrow$ read x and get value b

Processes: P1, P2, P3, P4 are different processors/computers/users.

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

以数据为中心的一致性模型

Data-centric consistency models

Sequential consistency

Definition

The result of any execution is the same as if the operations of all processes were executed in some sequential order, and the operations of each individual process appear in this sequence in the order specified by its program.

Figure (a) What happens?

Step 1 — P1 writes a

P1: W(x)a

Now x = a

Step 2 — P2 writes b

P2: W(x)b

Now x = b

Step 3 — Different processes see writes in SAME order

P3 reads:

R(x)b then R(x)a

P4 also reads:

R(x)b then R(x)a

So BOTH P3 and P4 observe: b first, then a

顺序一致性

定义

任何执行的结果与所有进程的操作按顺序执行相同，且每个进程的操作按程序指定的顺序出现。

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

Even if the order looks strange compared to actual write timing, all processors agree on the same order.

以数据为中心的一致性模型

Data-centric consistency models

Sequential consistency

顺序一致性

Figure (b) Now look carefully.

P3 reads: b then a

BUT P4 reads: a then b

Different processors observe DIFFERENT orders.

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

Why is this bad?

Because there is no single global order anymore.

P3 believes: b happened before a

P4 believes: a happened before b

System disagreement occurs.

This violates sequential consistency

Because sequential consistency requires:

All processes must observe writes in the same order.

Easy analogy

Two people checking Instagram comments:

User 1 sees: Comment B then A

User 2 sees: Comment A then B

Now users disagree about history.

That is inconsistency.

以数据为中心的一致性模型

Data-centric consistency models

Causal consistency

Definition

Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order by different processes.

因果一致性

定义

可能存在因果关系的写入必须被所有进程以相同顺序看到。不同进程可能以不同顺序显示并发写入。

P1:	W(x)a		
P2:	R(x)a	W(x)b	
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(a)

P1:	W(x)a		
P2:		W(x)b	
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

以数据为中心的一致性模型

Data-centric consistency models

Causal consistency

Figure (a) — CAUSAL RELATIONSHIP

Step 1

P1 writes: $W(x)a$

Step 2

P2 READS $aR(x)a$

So P2 has seen P1's write.

Step 3

Then P2 writes: $W(x)b$

This means:

b depends on a

because P2 created b AFTER seeing a .

This creates a causal relationship: $a \rightarrow b$ ("a happened before b")

Now look at readers

P3 sees: b then a

P4 sees: a then b

Why is P3 wrong?

Because: b causally depends on a

So nobody should see: b before a

That violates causal consistency.

因果一致性

P1: $W(x)a$

P2: $R(x)a$ $W(x)b$

P3: $R(x)b$ $R(x)a$

P4: $R(x)a$ $R(x)b$

(a)

P1: $W(x)a$

P2: $W(x)b$

P3: $R(x)b$ $R(x)a$

P4: $R(x)a$ $R(x)b$

(b)

Main idea of causal consistency

If one write depends on another:

then everyone must observe:

$a \rightarrow b$

a before b

以数据为中心的一致性模型

Data-centric consistency models

Causal consistency

Real-life analogy

Suppose:

Alice posts:

"Exam postponed"

Bob reads it and replies:

"Great news!"

You should NEVER see:

"Great news!"

before:

"Exam postponed"

because Bob's reply depends on Alice's post.

That is causal consistency.

因果一致性

现实生活中的类比

假设：

Alice发帖：

“考试推迟”

鲍勃读了信后回复：

“好消息！”

你绝对不应该看到：

“好消息！”

之前：

“考试推迟”

因为鲍勃的回复取决于爱丽丝的帖子。

这就是因果一致性。

以数据为中心的一致性模型

Data-centric consistency models

Causal consistency

Figure (b) — CONCURRENT WRITES

Now:

P1 writes: $W(x)a$

P2 independently writes: $W(x)b$

No read occurs between them. So:

a and b are concurrent

Neither depends on the other.

Now different orders are ALLOWED

P3 sees: b then a

P4 sees: a then b

This is acceptable.

Because there is NO causal dependency.

The writes are independent.

因果一致性

P1:	$W(x)a$		
P2:	$R(x)a$	$W(x)b$	
P3:		$R(x)b$	$R(x)a$
P4:		$R(x)a$	$R(x)b$

(a)

P1:	$W(x)a$		
P2:		$W(x)b$	
P3:		$R(x)b$	$R(x)a$
P4:		$R(x)a$	$R(x)b$

(b)

Main idea

Causally related writes

Must be seen in SAME order everywhere.

Concurrent writes

May be seen in DIFFERENT orders.

以数据为中心的一致性模型

Data-centric consistency models

Grouping operations

- ❑ Accesses to synchronization variables are sequentially consistent.
- ❑ No access to a synchronization variable is allowed to be performed until all previous writes have completed everywhere.
- ❑ No data access is allowed to be performed until all previous accesses to synchronization variables have been performed.

Basic idea

You don't care that reads and writes of a series of operations are immediately known to other processes. You just want the effect of the series itself to be known.

编组作业

- ❑ 对同步变量的访问是顺序一致的。
- ❑ 在所有之前的写入都完成之前，不允许访问同步变量。
- ❑ 在所有之前对同步变量的访问都完成之前，不允许进行任何数据访问。

基本理念

你不在乎一系列操作的读写会被其他进程立即知道。你只是想让系列本身的影响被大家知道。

以数据为中心的一致性模型

Data-centric consistency models

Grouping operations

Observation

Weak consistency implies that we need to lock and unlock data (implicitly or not).

Question

What would be a convenient way of making this consistency more or less transparent to programmers?

编组作业

观察

弱一致性意味着我们需要锁定和解锁数据（无论是隐含还是非）。

问题

有什么方便的方法能让程序员对这种一致性更加透明或更透明？

P1:	Acq(Lx)	W(x)a	Acq(Ly)	W(y)b	Rel(Lx)	Rel(Ly)
P2:			Acq(Lx)	R(x)a		R(y) NIL
P3:				Acq(Ly)	R(y)b	

- ☐ P1 acquires Lx, writes x = a then acquires Ly, writes y = b then releases both locks
- ☐ This is good because the two writes are treated as a group.
- ☐ That means other processes, like P2 and P3, cannot see only part of the update in a confusing way.

以客户端为中心的一致性模型

Client-centric consistency models

Overview

- ☐ System model
- ☐ Monotonic reads
- ☐ Monotonic writes
- ☐ Read-your-writes
- ☐ Write-follows-reads

Goal

Show how we can perhaps avoid systemwide consistency, by concentrating on what specific clients want, instead of what should be maintained by servers.

概述

- ☐ 系统模型
- ☐ 单调读段
- ☐ 单调写入
- ☐ 读写
- ☐ 写-跟-读

目标

展示我们如何通过关注特定客户端的需求，而非服务器应维护的内容，避免系统范围的一致性。

以客户端为中心的一致性模型

Client-centric consistency models

Consistency for mobile users

Example

Consider a distributed database to which you have access through your notebook. Assume your notebook acts as a front end to the database.

- ❑ At location A you access the database doing reads and updates.
- ❑ At location B you continue your work, but unless you access the same server as the one at location A, you may detect inconsistencies:
 - ❑ your updates at A may not have yet been propagated to B
 - ❑ you may be reading newer entries than the ones available at A
 - ❑ your updates at B may eventually conflict with those at A

移动用户的一致性

示例

考虑一个分布式数据库，你可以通过笔记本访问它。假设你的笔记本是数据库的前端。

- ❑ 在地点A，你访问数据库进行读取和更新。
- ❑ 在地点B你继续工作，但除非你访问与地点A相同的服务器，否则可能会发现不一致：
 - ❑ 你在A的更新可能还没有传播到B。
 - ❑ 你可能正在阅读比A页面上可用的条目更新的条目
 - ❑ 你在B的更新最终可能会与A的更新冲突

以客户端为中心的一致性模型

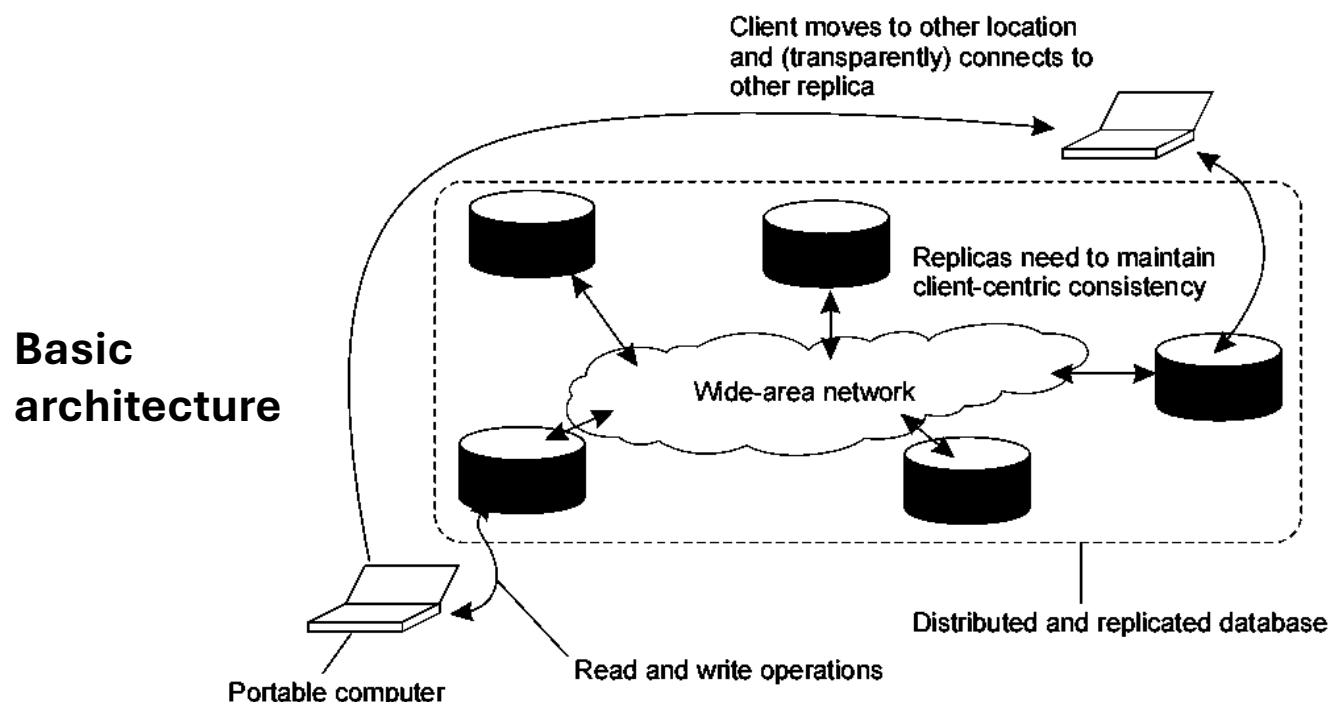
Client-centric consistency models

Consistency for mobile users

The only thing you really want is that the entries you updated and/or read at A, are in B the way you left them in A. In that case, the database will appear to be consistent to you.

移动用户的一致性

你真正想要的，是你在A处更新和/或阅读的条目，在B中保持在A时的状态。在这种情况下，数据库看起来会是一致的。



以客户端为中心的一致性模型

Client-centric consistency models

Monotonic reads

If a process reads the value of a data item x , any successive read operation on x by that process will always return that same or a more recent value.

Suppose x is a user profile version:

First read: x = version 3

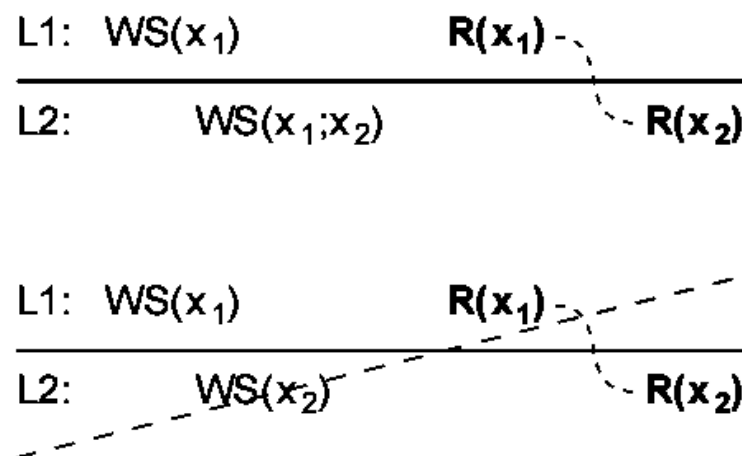
Later read: you may see version 3 again, or version 4

But you should not later see version 2

That would be confusing because the system would look like it “went back in time.”

单调读段

如果进程读取数据项 x 的值，该进程对 x 的任何后续读取操作都会返回相同或较新的值。



以客户端为中心的一致性模型

Client-centric consistency models

The figure shows two replicas of the same data:

L1

L2

Each replica may have a different version of x .

Top part

L1: $WS(x_1)$ means replica L1 stores version x_1

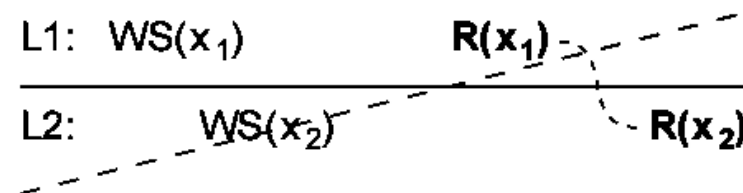
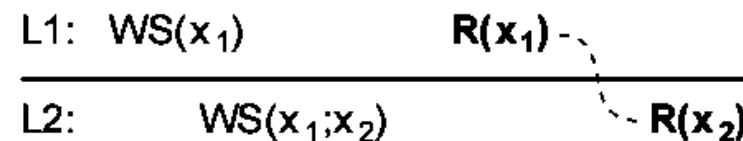
L2: $WS(x_1, x_2)$ means replica L2 stores a newer version x_2 in addition to x_1

Then:

The process reads $R(x_1)$ from L1

Later it reads $R(x_2)$ from L2

This is good for monotonic reads because the second read is same or newer.



以客户端为中心的一致性模型

Client-centric consistency models

Bottom part of the figure

The bottom part shows a situation where the process may first read from a more recent place and then later from an older one.

This can cause trouble.

For example:

first read: $R(x_1)$ from a replica that still has old data

later read: $R(x_2)$ from another replica with newer data

That is okay.

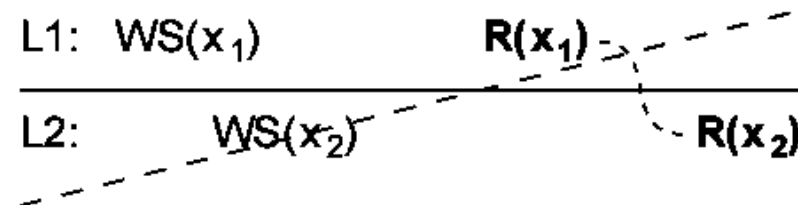
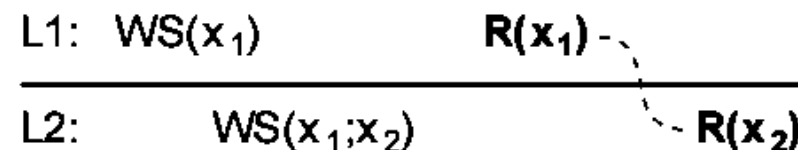
But the problem would be if the process later switched to a replica with an older value than the one it already saw.

Then it would feel like:

first read: version 4

later read: version 2

That breaks monotonic reads.



Imagine checking a news article:

At 10:00 you read: "Score is 1-0"

At 10:05 you refresh and see: "Score is 2-0"

That is fine.

But if at 10:10 you see again: "Score is 1-0", that is not monotonic and is very confusing.

以客户端为中心的一致性模型

Client-centric consistency models

单调读段

示例

自动读取来自不同服务器的个人日历更新。单调读取保证用户无论自动读取来自哪个服务器，都能看到所有更新。

示例

在你移动时阅读（不修改）来信。

每次你连接到不同的电子邮件服务器时，该服务器至少会和你之前访问过的服务器获取所有更新。

Monotonic reads

Example

Automatically reading your personal calendar updates from different servers.

Monotonic Reads guarantees that the user sees all updates, no matter from which server the automatic reading takes place.

Example

Reading (not modifying) incoming mail while you are on the move.

Each time you connect to a different e-mail server, that server fetches (at least) all the updates from the server you previously visited.

以客户端为中心的一致性模型

Client-centric consistency models

Monotonic writes

A write operation by a process on a data item x is completed before any successive write operation on x by the same process.

Think of it like this:

If you write:

$x = 1$

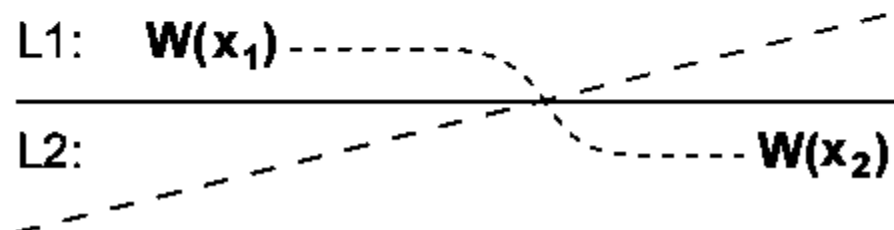
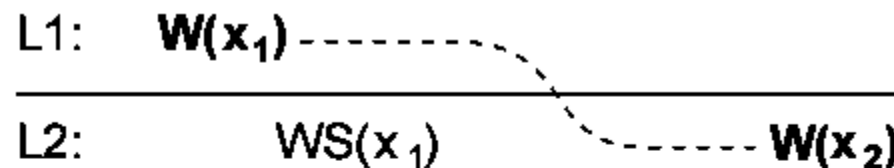
then $x = 2$

the system must not let the second write happen “too early” or appear before the first one.

So everyone should see the writes in the correct order.

单调写入

进程对数据项 x 的写入操作，会在同一进程对 x 进行任何后续写入操作之前完成。



以客户端为中心的一致性模型

Client-centric consistency models

Monotonic writes

Top figure

The process writes x_1 at L1

That write is completed

Then it writes x_2 at L2

This is correct because the writes happen in order:

$$W(x_1) \rightarrow W(x_2)$$

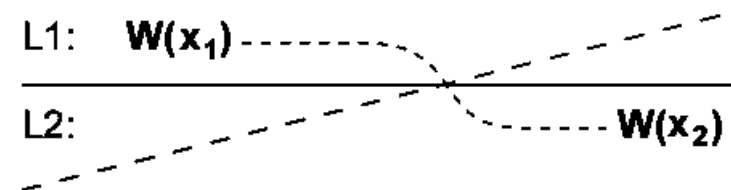
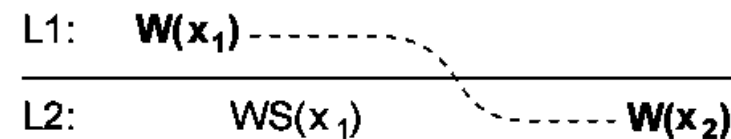
So monotonic writes are preserved.

The dashed line shows the order in which the writes travel or are applied across replicas.

The important rule is:

the second write should not be applied before the first one is done

单调写入



The figure shows two replicas:

L1

L2

And writes from the same process:

$W(x_1)$ = write value x_1

$W(x_2)$ = later write value x_2

以客户端为中心的一致性模型

Client-centric consistency models

Monotonic writes

Bottom figure

The bottom figure shows a similar idea:

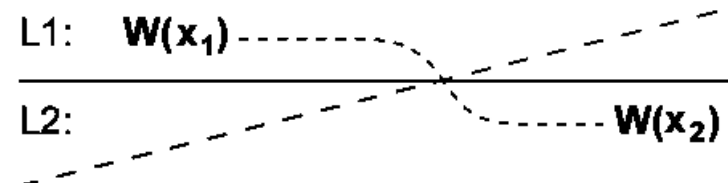
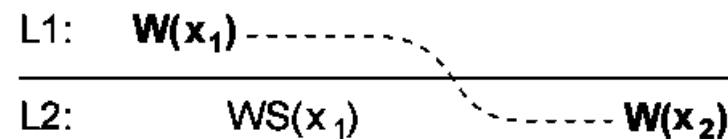
the process writes $W(x_1)$
then later writes $W(x_2)$

Even if the writes go to different replicas, the system must make sure:

x_1 is fully written first
only then x_2

So the write sequence stays consistent.

单调写入



The figure shows two replicas:

L1

L2

And writes from the same process:

$W(x_1)$ = write value x_1

$W(x_2)$ = later write value x_2

以客户端为中心的一致性模型

Client-centric consistency models

Monotonic writes

Easy real-life example

Imagine you are editing your status:

First you change it to: “Busy”

Then you change it to: “In a meeting”

Monotonic writes means other people should not see:

“In a meeting” first

and then later “Busy”

That would be backwards and confusing.

bank balance update

Suppose a process does:

deposit \$50

then deposit \$20

The system should apply them in order:

balance increases by \$50 first

then by \$20

It should not reverse the order.

单调写入

简单的现实例子

想象你正在编辑你的状态:

首先你把它改成: “忙碌”

然后你改成: “正在开会”

单调书写意味着别人不应看到:

先说“在开会中”

后来又是《忙碌》

那样会很反复且令人困惑。

银行余额更新

假设一个过程满足:

押金50美元

然后存入20美元

系统应按以下顺序应用:

余额先增加50美元

然后是20美元

它不应该逆转这个顺序。

以客户端为中心的一致性模型

Client-centric consistency models

Read your writes

The effect of a write operation by a process on data item x , will always be seen by a successive read operation on x by the same process.

Top part

The process writes $W(x_1)$ on L1

Later, the process reads $R(x_2)$ from L2

Here, x_2 is the value that includes the effect of the previous write.

Bottom part

This also shows the process writing on one replica and then reading from another replica.

Even if the read happens at a different place, the system must make sure the process still sees the effect of its own write.

So the dashed line means:

your write must be visible to
your next read

阅读你的文章

进程对数据项 x 的写入操作，总会被同一进程对 x 的后续读取操作所体现。

Example(s)

Updating your Web page and guaranteeing that your Web browser shows the newest version instead of its cached copy.

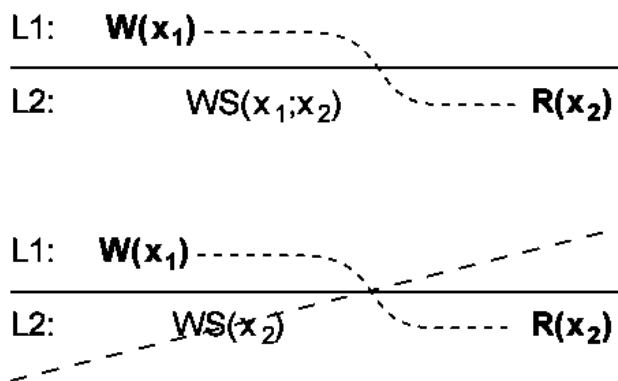
示例

更新你的网页，确保浏览器显示最新版本，而不是缓存的副本。

you write $x = 5$

then later you read x

you should see 5, not an older value like 3

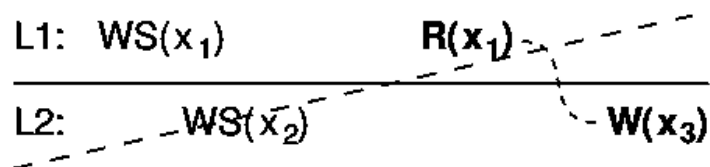
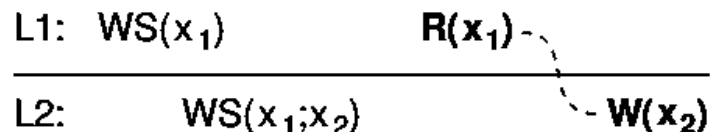


以客户端为中心的一致性模型

Client-centric consistency models

Writes follow reads

A write operation by a process on a data item x following a previous read operation on x by the same process, is guaranteed to take place on the same or a more recent value of x that was read.



写入跟随读

进程在同一进程对 x 的读取操作后，对数据项 x 进行写操作，必定会在相同或较新的 x 值上执行。

Example

Suppose you read a message count:

First read:

$x = 10$

Then you add 1 and write back:

Write: $x = 11$

That is correct because your write follows what you just read.

以客户端为中心的一致性模型

Client-centric consistency models

Writes follow reads

写入跟随读

The figure again shows two replicas:

L1

L2

And values:

x_1 = old value

x_2 = newer value

x_3 = even newer value

Top part

The process reads $R(x_1)$ from L1

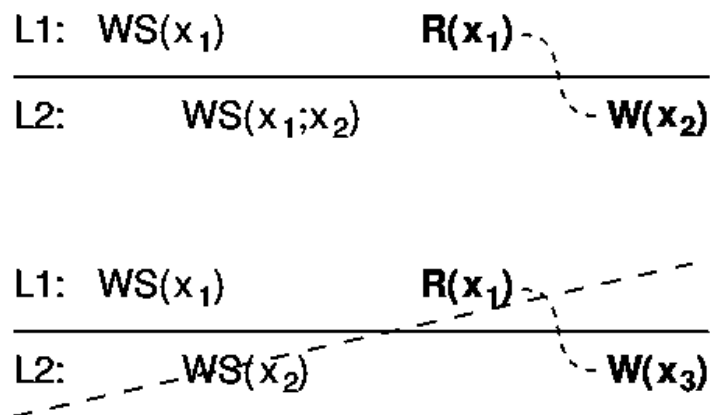
Then it writes $W(x_2)$ on L2

This is okay because the write is based on the value that was read, or a newer one.

Bottom part

Here the process reads $R(x_1)$ and later writes $W(x_3)$.

That is also okay because x_3 is newer than x_1 .



You open a form and read the current address:

$x = \text{"Old Street"}$

Then you edit it to:

$x = \text{"New Street"}$

That write follows your read.

But if the server already changed it to something newer, your update should not go backward.

The key point is:

your write should not be based on something older than what you already read

非常简短的对比

Very short comparison

- ❑ Monotonic reads: once you read a value, you shouldn't later see an older one
 - ❑ Monotonic writes: your writes should happen in order
 - ❑ Read your writes: after you write something, you should be able to read it yourself
 - ❑ Writes follow reads: after you read, your write should respect that read
- ❑ 单调读法：一旦你读到一个数值，之后就不应该再看到更早的值了
 - ❑ 单调写作：写作应按顺序进行
 - ❑ 阅读你的写作：写完东西后，你应该能自己阅读
 - ❑ 写入随读：阅读后，写入应尊重阅读

非常简短的对比

Very short comparison

When to choose which Consistency Model?

Strong Consistency: Every read reflects the latest write. Use it when the most recent data is critical, like banking or inventory systems.

Sequential Consistency: Operations appear in a predictable order, but not globally synchronized. Useful when order matters but full synchronization isn't needed.

Causal Consistency: Related operations are seen in the correct order, but unrelated operations may vary. Ideal for social media or user interaction systems.

Weak Consistency: Data may be temporarily out of sync, prioritizing availability and performance. Suitable when real-time consistency isn't essential.

Monotonic Reads/Writes: User's view of data never goes backward in time. Ensures sequential actions remain consistent and avoids outdated information.

复制管理

Replica Management

Distribution protocols

- ❑ Replica server placement
- ❑ Content replication and placement
- ❑ Content distribution

分发协议

- ❑ 副本服务器的部署
- ❑ 内容复制与放置
- ❑ 内容分发

复制管理

Replica Management

Replica placement

Essence

Figure out what the best K places are out of N possible locations.

- ☐ Select best location out of N – K for which the average distance to clients is minimal. Then choose the next best server. (Note: The first chosen location minimizes the average distance to all clients.)
- ☐ Computationally expensive.
- ☐ Select the K-th largest autonomous system and place a server at the best-connected host. Computationally expensive.
- ☐ Position nodes in a d-dimensional geometric space, where distance reflects latency. Identify the K regions with highest density and place a server in every one. Computationally cheap.

复制管理

Replica Management

Content replication

Distinguish different processes

A process is capable of hosting a replica of an object or data:

- ❑ **Permanent replicas:** Process/machine always having a replica
- ❑ **Server-initiated replica:** Process that can dynamically host a replica on request of another server in the data store
- ❑ **Client-initiated replica:** Process that can dynamically host a replica on request of a client (**client cache**)

内容复制

区分不同的过程

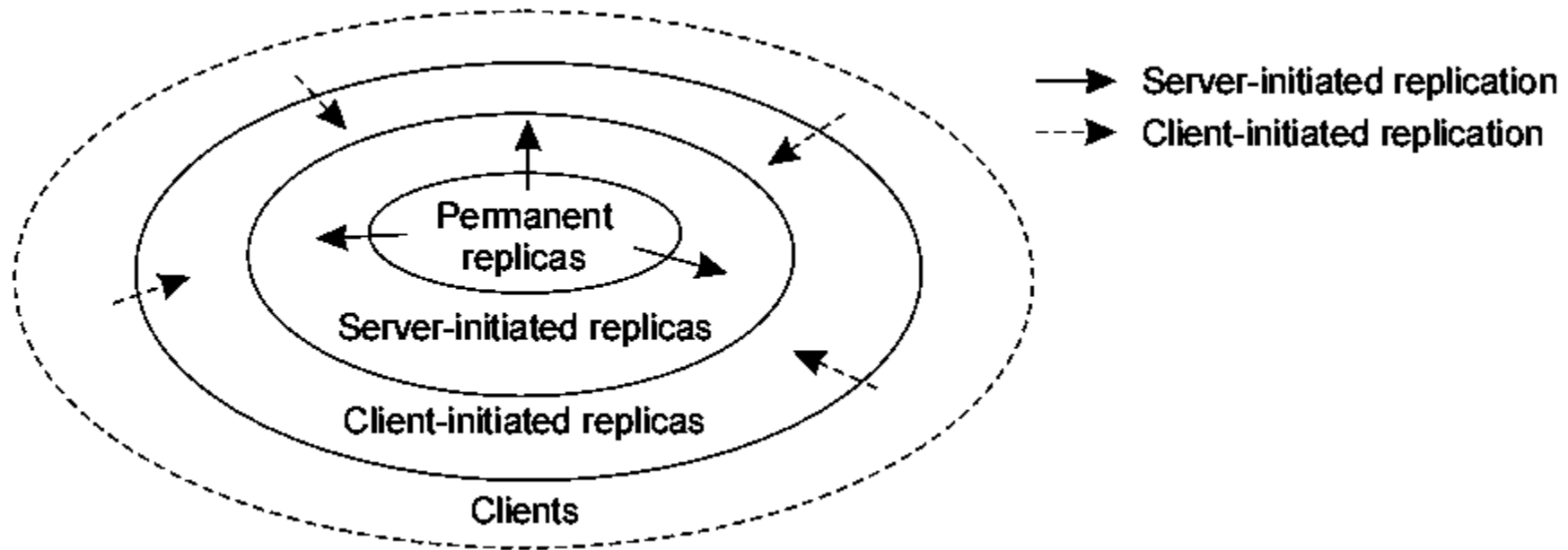
进程能够承载对象或数据的副本:

- ❑ **永久副本:** 进程/机器始终拥有副本
- ❑ **服务器发起副本:** 能够在数据存储中另一服务器请求时动态托管副本的进程
- ❑ **客户端发起副本:** 能够在客户端请求时动态托管副本的进程 (**客户端缓存**)

复制管理

Replica Management

Content replication

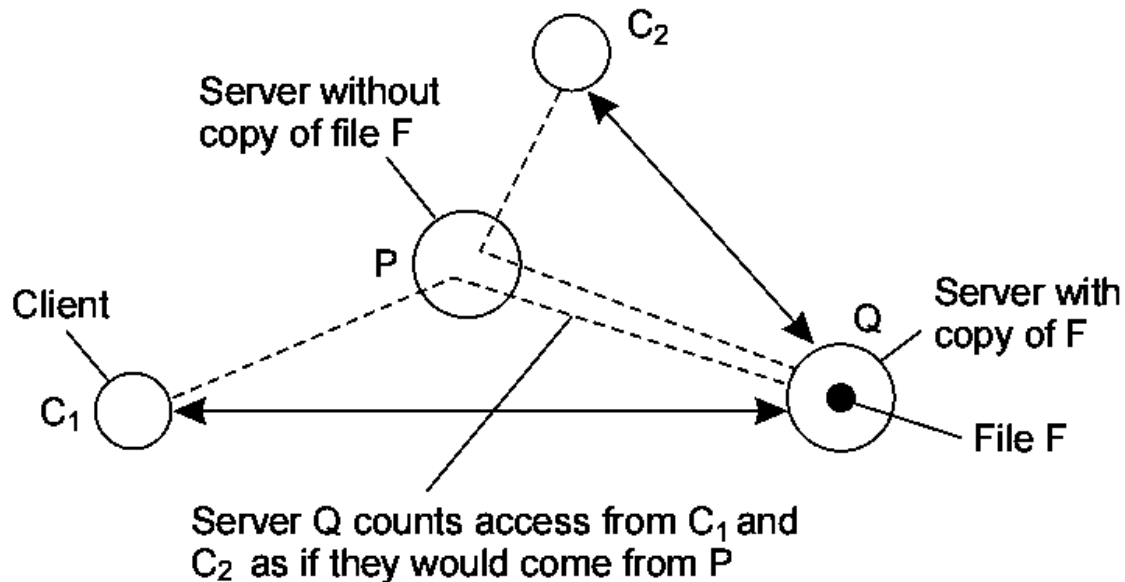


复制管理

Replica Management

Server-initiated replicas

- ❑ Keep track of access counts per file, aggregated by considering server closest to requesting clients
- ❑ Number of accesses drops below threshold $D \Rightarrow$ drop file
- ❑ Number of accesses exceeds threshold $R \Rightarrow$ replicate file
- ❑ Number of access between D and $R \Rightarrow$ migrate file



There are two servers:

P = server without a copy of file F

Q = server with a copy of file F

And two clients: C1 and C2

The dashed lines show clients accessing the file through their nearby server.

Important idea in the figure

Server Q counts requests from C1 and C2 as if they came from P.

That means Q is saying:

“These requests are really about file access near P, so I’ll use that to decide whether file F should stay here, move, or be copied.”

复制管理

Replica Management

Content distribution

Model

Consider only a client-server combination:

- ☐ Propagate only notification/invalidation of update (often used for caches)
- ☐ Transfer data from one copy to another (distributed databases): **passive replication**
- ☐ Propagate the update operation to other copies: active replication

Note

No single approach is the best, but depends highly on available bandwidth and read-to-write ratio at replicas.

复制管理

Replica Management

Content distribution: client/server system

- ☐ Pushing updates: server-initiated approach, in which update is propagated regardless whether target asked for it.
- ☐ Pulling updates: client-initiated approach, in which client requests to be updated.

Issue	Push-based	Pull-based
1:	List of client caches	None
2:	Update (and possibly fetch update)	Poll and update
3:	Immediate (or fetch-update time)	Fetch-update time
1: State at server		
2: Messages to be exchanged		
3: Response time at the client		

复制管理

Replica Management

Content distribution

Observation

We can dynamically switch between pulling and pushing using leases:

A contract in which the server promises to push updates to the client until the lease expires.

Issue

Make lease expiration time dependent on system's behavior (adaptive leases):

- ❑ **Age-based leases:** An object that hasn't changed for a long time, will not change in the near future, so provide a long-lasting lease
- ❑ **Renewal-frequency based leases:** The more often a client requests a specific object, the longer the expiration time for that client (for that object) will be
- ❑ **State-based leases:** The more loaded a server is, the shorter the expiration times become

一致性协议

Consistency protocols

Consistency protocol

Describes the implementation of a specific consistency model.

- ☐ Continuous consistency
- ☐ Primary-based protocols
- ☐ Replicated-write protocols

一致性协议

描述特定一致性模型的实现。

- ☐ 连续一致性
- ☐ 基于主协议
- ☐ 复制写协议

一致性协议

Consistency protocols

Continuous consistency: Numerical errors

Principal operation

Every server S_i has a log, denoted as $\log(S_i)$.

Consider a data item x and let $\text{weight}(W)$ denote the numerical change in its value after a write operation W . Assume that

$$\forall W : \text{weight}(W) > 0$$

W is initially forwarded to one of the N replicas, denoted as $\text{origin}(W)$. $TW[i, j]$ are the writes executed by server S_i that originated from S_j :

$$TW[i, j] = \sum \{ \text{weight}(W) \mid \text{origin}(W) = S_j \ \& \ W \in \log(S_i) \}$$

一致性协议

Consistency protocols

Continuous consistency: Numerical errors

Note

Actual value $v(t)$ of x :

$$v(t) = v_{init} + \sum_{k=1}^N TW[k, k]$$

value v_i of x at replica i :

$$v_i = v_{init} + \sum_{k=1}^N TW[i, k]$$

一致性协议

Consistency protocols

Continuous consistency: Numerical errors

Problem

We need to ensure that $v(t) - v_i < \delta_i$ for every server S_i .

Approach

Let every server S_k maintain a view $TW_k[i, j]$ of what it believes is the value of $TW[i, j]$. This information can be gossiped when an update is propagated.

Note

$$0 \leq TW_k[i, j] \leq TW[i, j] \leq TW[j, j]$$

一致性协议

Consistency protocols

Continuous consistency: Numerical errors

Solution

S_k sends operations from its log to S_i when it sees that $TW_k[i, k]$ is getting too far from $TW[k, k]$, in particular, when

$$TW[k, k] - TW_k[i, k] > \delta_i / (N - 1)$$

Question

To what extent are we being pessimistic here: where does $\delta_i / (N - 1)$ come from?

Note

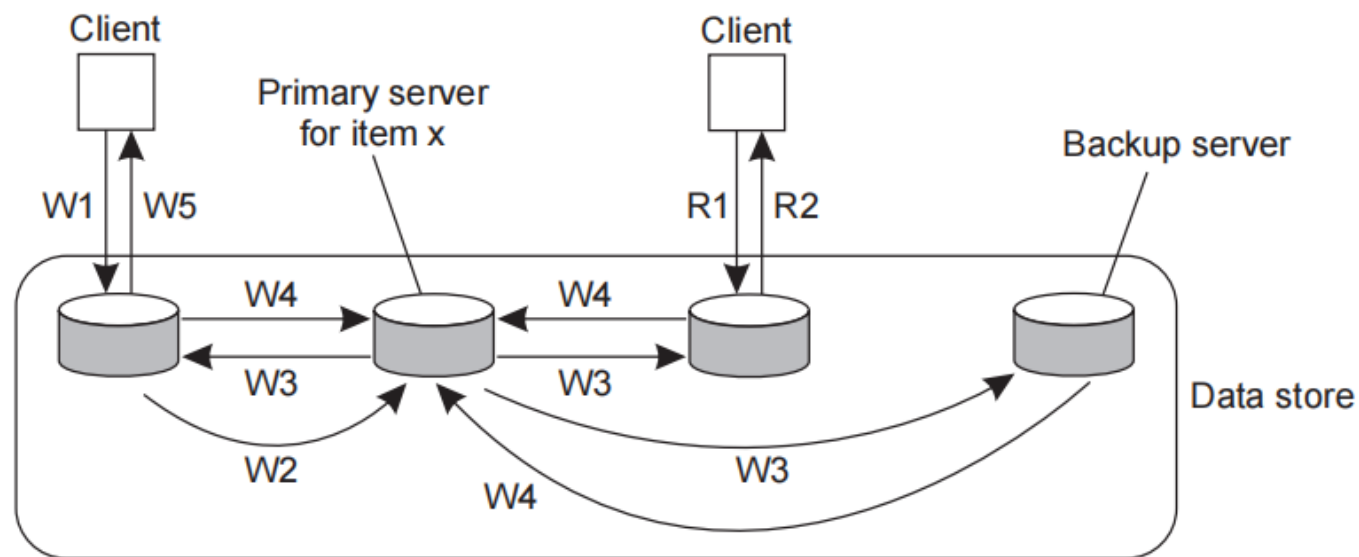
Staleness can be done analogously, by essentially keeping track of what has been seen last from S_i .

一致性协议

Consistency protocols

Primary-based protocols

Primary-backup protocol



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

一致性协议

Consistency protocols

Primary-based protocols

Example primary-backup protocol

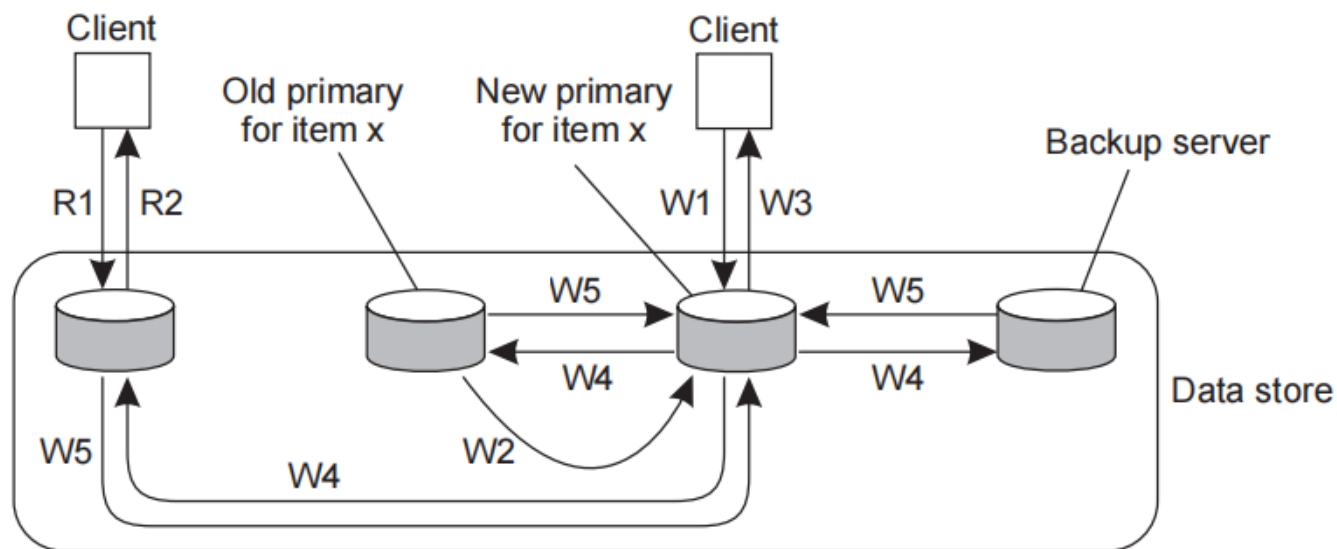
Traditionally applied in distributed databases and file systems that require a high degree of fault tolerance. Replicas are often placed on same LAN.

一致性协议

Consistency protocols

Primary-based protocols

Primary-backup protocol with local writes



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

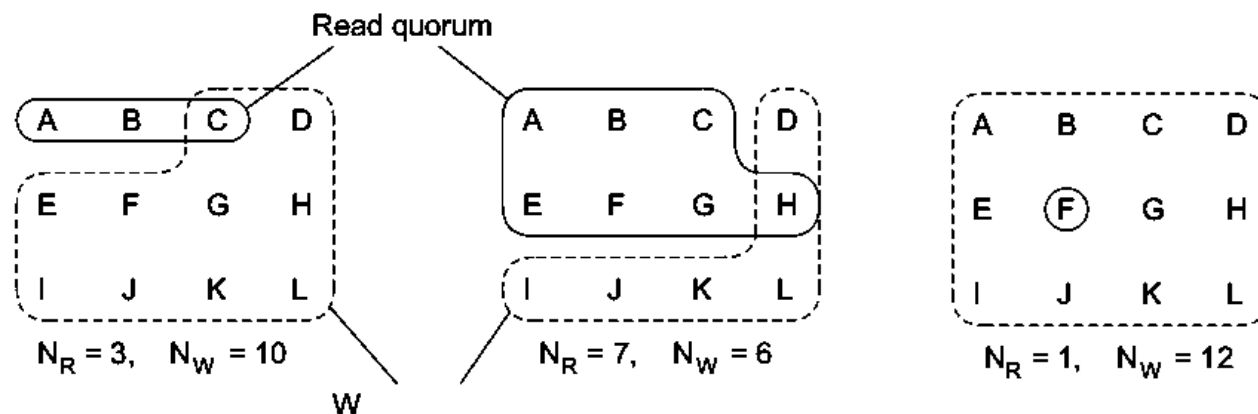
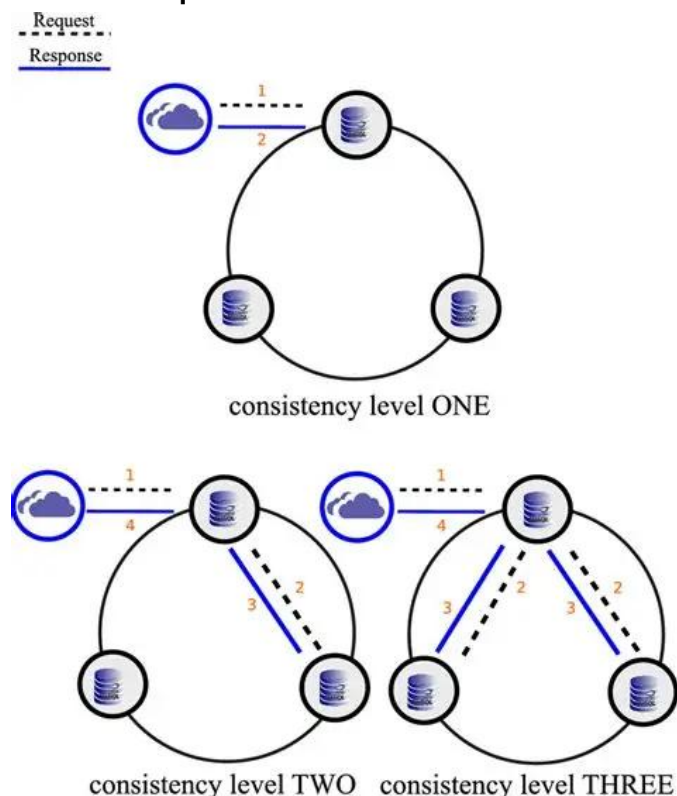
一致性协议

Consistency protocols

Replicated-write protocols

Quorum-based protocols

Ensure that each operation is carried out in such a way that a majority vote is established: distinguish read quorum and write quorum:



required: $N_R + N_W > N$ and $N_W > N/2$